

CHOICE Forge — Layer 4 Explainer

What the polarity guard actually does: a rule-based, untrained runtime backstop that catches T5's negation failures with direction checks, template corrections, and a flag-not-guess fallback.

Source line from the architecture doc: [Layer 4] `polarity_guard.py` (rule-based, runtime only, no training) → catches a specific failure mode T5-small has on negated constraints and either fixes it with a template or flags it instead of silently returning a wrong answer.

LAYER 4 • CONCEPT 1

Rule-based runtime guardrail (why this layer exists, and why it's not "training")

In plain terms

Layers 2 and 3 are like two employees who learned their jobs from examples and occasionally make mistakes. Layer 4 is a supervisor who doesn't do the work itself — it just double-checks the finished output against a small set of common-sense rules and steps in only when something looks obviously wrong.

Technically

Unlike the CRF and T5 (both fit weights to the 100 labeled rows via gradient-based optimization), `polarity_guard.py` contains zero learned parameters — it's pure Python control flow (regex matching, string checks) that runs once per prediction, after generation, with no training step and no dependency on dataset size. This is deliberate: catch a *known, narrow, well-understood* failure mode with a rule, rather than hoping more training data eventually fixes it — a cheap, deterministic patch for a problem whose root cause (23/655 negation examples) is already diagnosed.

```
# no .fit(), no gradient descent, no model file to load --
# this function is called fresh on every prediction
def check_and_correct(source_text, generated_value):
    ...
```

Why it matters in this pipeline

This is the difference between “the model is 96% accurate so we accept the 4%” and “we know exactly which 4% is likely wrong, and we catch it deterministically.” It's a targeted fix for a diagnosed problem, not a general-purpose accuracy booster.

LAYER 4 • CONCEPT 2

Direction detection with negation-flipping

In plain terms

Figure out whether a phrase is pushing a number up or down — “increase,” “raise,” “more” mean up; “cut,” “reduce,” “under” mean down — but if there's a “without” or “not” sitting in front, flip the answer, because

“without increasing” actually means “keep it down.”

Technically

`direction()` scans for increase/decrease vocabulary, then checks `has_negation()` on the same text and flips the result if a negation cue is present. This is applied to **both** the source span and the generated value, and the two results get compared.

```
def direction(text):
    """Return 'increase', 'decrease', or 'neutral', accounting for negation flip."""
    if not text:
        return "neutral"
    t = text.lower()
    neg = has_negation(t)
    inc = any(w in t for w in INCREASE_WORDS)
    dec = any(w in t for w in DECREASE_WORDS)
    if inc and not dec:
        base = "increase"
    elif dec and not inc:
        base = "decrease"
    else:
        return "neutral"
    return ("decrease" if base == "increase" else "increase") if neg else base
```

Why it matters in this pipeline

This is what catches the exact bug reported earlier — `direction("without increasing the ad spend budget") → "decrease"`, `direction("ad spend budget raised") → "increase"`. Disagreement flagged.

Negation-drop detection (a second, independent trigger)

In plain terms

Sometimes the model doesn't flip the meaning, it just drops the “without”/“not” entirely and writes the positive version as if the negation was never there. Direction-checking alone won't catch that if neither version uses an increase/decrease word at all.

Technically

"without using external agencies" → "use external agencies" contains no increase/decrease vocabulary on either side, so `direction()` returns "neutral" for both and the mismatch check alone passes it through. `check_and_correct()` runs a second, independent check: does the source contain a negation cue that's absent from the generated value entirely?

```
src_dir = direction(source_text)
gen_dir = direction(generated_value)
direction_mismatch = (src_dir != "neutral" and gen_dir != "neutral"
                      and src_dir != gen_dir)
negation_dropped = has_negation(source_text) and not has_negation(generated_value)

if not direction_mismatch and not negation_dropped:
    return generated_value, False, False # nothing wrong, pass through
```

Why it matters in this pipeline

This was a real gap found by actually testing the guard against a case outside the two originally reported — direction-only logic would have silently let “use external agencies” through as correct.

Template-based correction (why rules work well specifically here)

In plain terms

Constraint phrases in this dataset follow a small number of very regular patterns — “without X-ing Y” almost always should become “must not X Y.” Since the pattern is so consistent, a fixed rule can fix it correctly almost every time, without needing hundreds of examples the way the T5 model would.

Technically

TEMPLATE_OVERRIDES is an ordered list of regex-pattern → rewrite-function pairs, tried in sequence. The gerund conversion (“increasing” → “increase”) uses an explicit lexicon rather than a guessed suffix rule, because English gerund-to-base-verb formation is irregular enough (using→use, but running→run) that blind stripping silently produces wrong words — unknown gerunds fall through to a safer generic template that keeps the gerund form rather than mangling it.

```
GERUND_LEXICON = {
    "increasing": "increase", "raising": "raise", "using": "use",
    "losing": "lose", "closing": "close", "adding": "add", ...
}
```

```

TEMPLATE_OVERRIDES = [
    (re.compile(r"without\s+(\w+ing)\s+(.+)", re.I),
     lambda m: f"must not {GERUND_LEXICON[m.group(1).lower()]} {m.group(2)}".strip()
     if m.group(1).lower() in GERUND_LEXICON else None), # None = skip, try next
    (re.compile(r"(?:spending\s+)?no more than\s+(.+)", re.I),
     lambda m: f"capped at {m.group(1)}"),
    (re.compile(r"without\s+(.+)", re.I),
     lambda m: f"must not {m.group(1)}"), # safe fallback, keeps gerund form
]

```

Why it matters in this pipeline

This is the same tension raised at the very start of this project — “hand-written rules don't scale to every sentence” — but constraint negation is exactly the narrow, regular exception where that criticism doesn't apply. It's not a rule per sentence, it's three rules covering the actual patterns present in the data.

Flag instead of guess (the fallback, and confidence composition)

In plain terms

If a mismatch is found but none of the fixed templates apply, the guard doesn't try to be clever and guess a fix — it leaves the model's answer as-is but marks it “don't trust this one without a human check,” and lowers its confidence score so that flag is visible downstream.

Technically

`check_and_correct()` returns a 3-tuple — (`value`, `was_corrected`, `flagged_for_review`). `pipeline.py` uses all three: a template correction caps confidence at 0.7 (it's a rule, not the model's own judgment, so it shouldn't look as confident as an unflagged model output); an unresolved flag caps confidence at 0.3 and sets `flagged_for_review: true` in the final row.

```
corrected_value, was_corrected, flagged = check_and_correct(joined_span, value)
confidence = extraction_conf * synth_conf
if was_corrected:
    confidence = min(confidence, 0.7)
if flagged:
    confidence = min(confidence, 0.3)
```

Why it matters in this pipeline

This is the design principle that makes the whole pipeline trustworthy for downstream use — a bad answer with a visible low-confidence flag is a manageable problem; a bad answer that looks exactly as confident as a good one is the actual danger for anything reading this table automatically.

Quick reference

Concept	Where it lives	Input	Output
Runtime guardrail	<code>polarity_guard.py</code> (whole module)	source span + generated value	pass-through or correction
Direction detection	<code>direction()</code>	any text span	'increase' / 'decrease' / 'neutral'
Negation-drop check	<code>has_negation()</code> comparison	source vs. generated value	boolean trigger
Template correction	<code>TEMPLATE_OVERRIDES</code> / <code>_try_templates()</code>	matched source pattern	rewritten value or None
Flag + confidence cap	<code>check_and_correct()</code> -> <code>pipeline.py</code>	correction outcome	<code>flagged_for_review</code> + capped confidence